



5장. 합성곱 신경망 I

4팀 김예나 이가은 김규리

목차

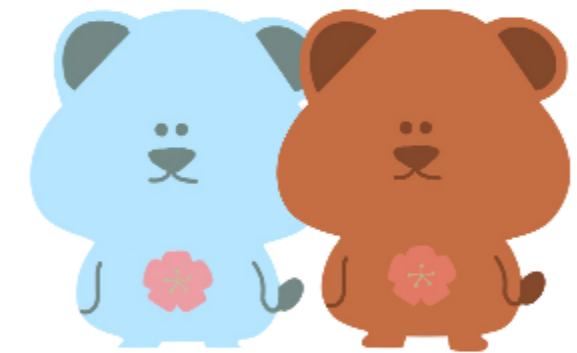
#01 전이학습(1)

#02 전이학습(2)

#03 설명 가능한 CNN



전이학습 (1)



#01 전이학습 (1)

#1 전이 학습의 개념과 필요성

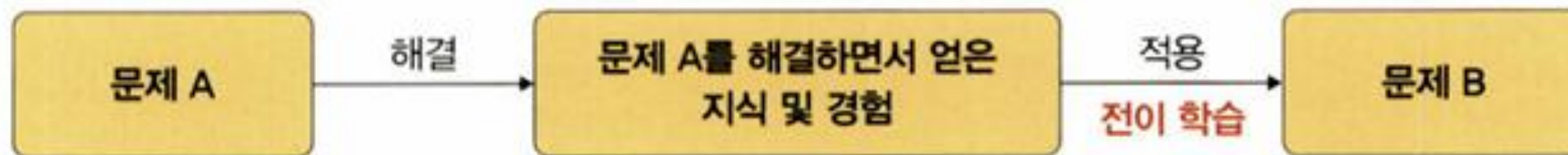
#2 전이 학습 방법 : 특성 추출 기법

#3 특성 추출 기법 코드 설명

#01 전이학습의 개념과 필요성

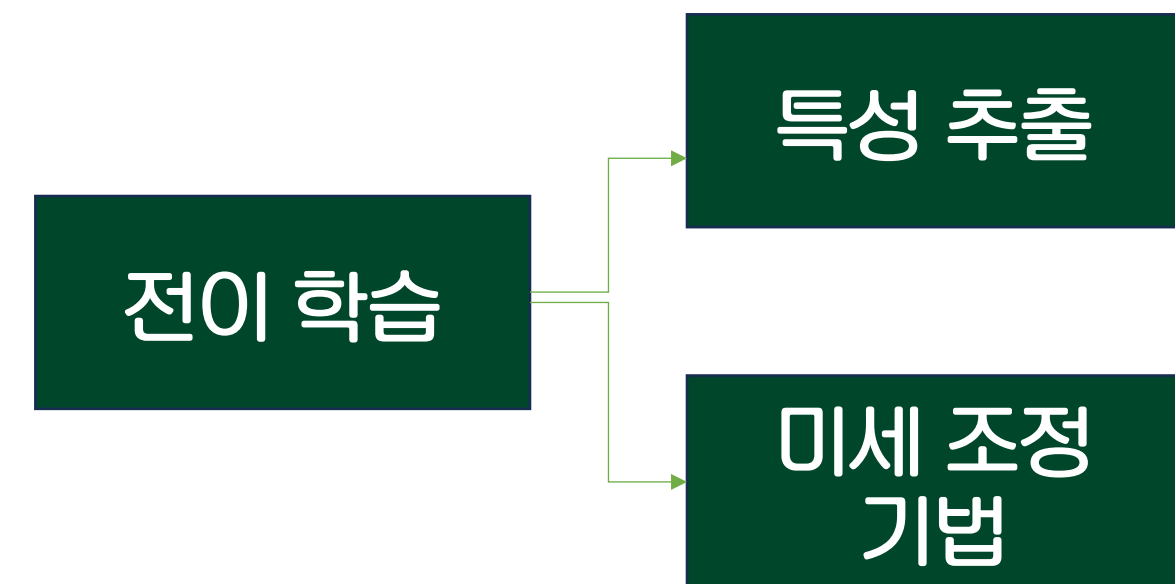
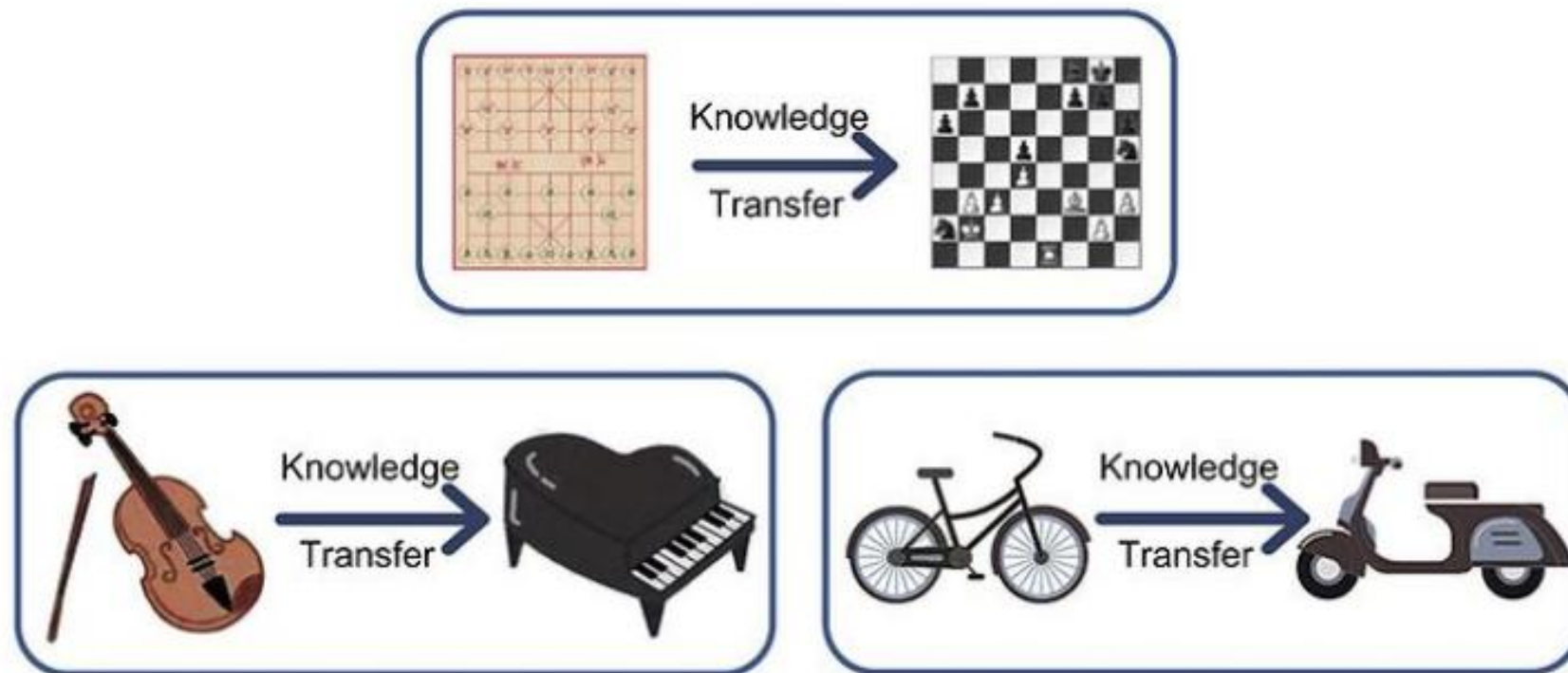
- 합성곱 신경망 기반의 딥러닝 모델 훈련의 문제점 : 큰 데이터셋 확보 어려움
- **전이 학습** : 이미지넷(ImageNet)과 같이 큰 데이터셋을 써서 훈련된 모델(사전 훈련된 모델)의 가중치를 가져와 해결하고자 하는 과제에 맞게 보정해서 사용하는 것
⇒ 비교적 적은 수의 데이터로도 원하는 과제 해결 가능

▼ 그림 5-29 전이 학습



#01 전이학습의 개념과 필요성

- 합성곱 신경망 기반의 딥러닝 모델 훈련의 문제점 : 큰 데이터셋 확보 어려움
- **전이 학습** : 이미지넷(ImageNet)과 같이 큰 데이터셋을 써서 훈련된 모델(사전 훈련된 모델)의 가중치를 가져와 해결하고자 하는 과제에 맞게 보정해서 사용하는 것
⇒ 비교적 적은 수의 데이터로도 원하는 과제 해결 가능

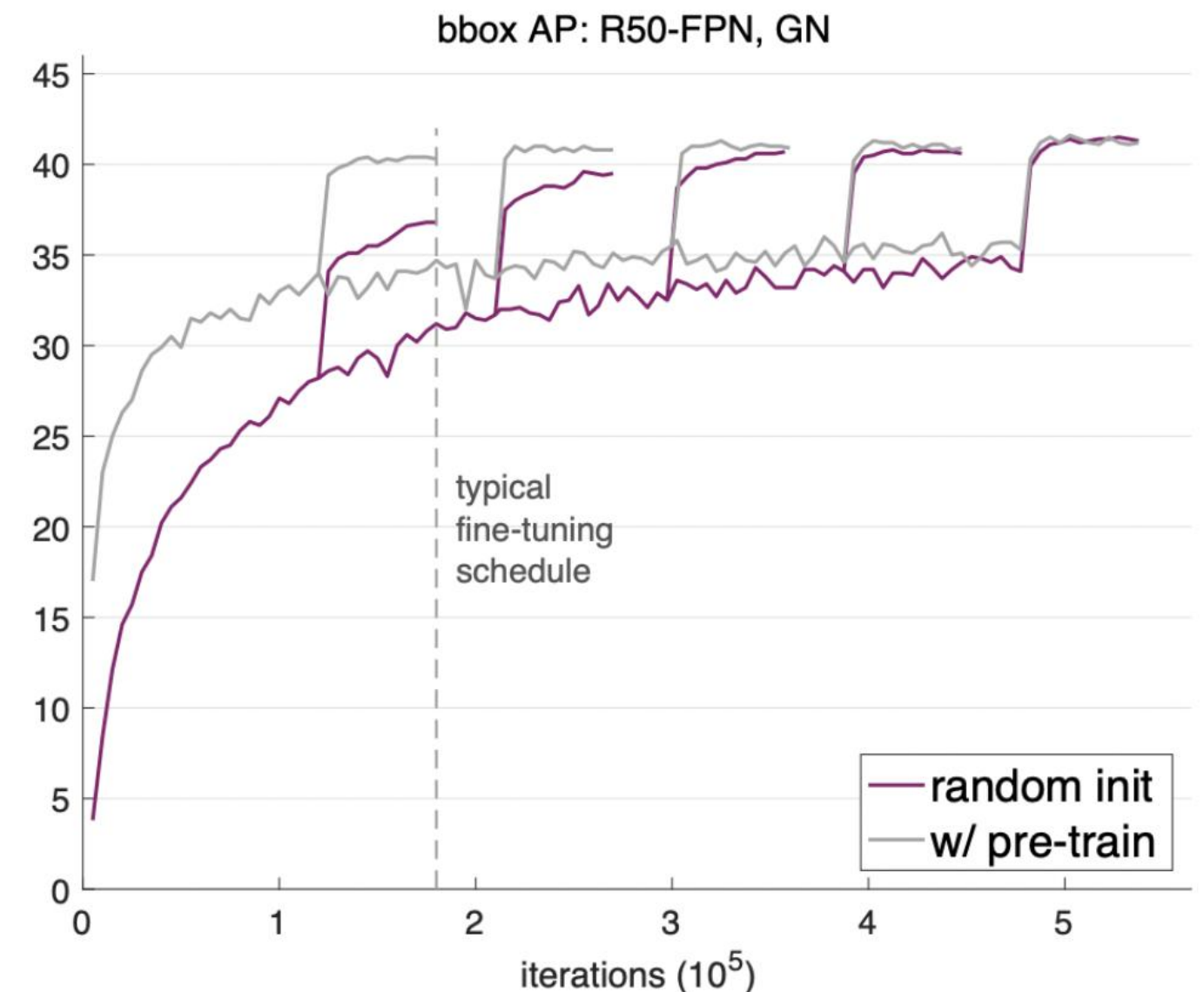


#01 전이학습의 개념과 필요성

필요성

- 전체 모델을 처음부터 설계하고 훈련하는 데에는 많은 시간과 비용이 소모됨
- 전이학습 : 시간, 비용 절약 및 성능 유지 가능
- > 실제로 2018년 FAIR(Facebook AI Research) 논문에서 실험을 통해 전이 학습이 학습 속도 면에서 효과가 있음을 증명

- 동일한 조건에서 사전학습 모델을 사용하였을 때보다, 처음부터 훈련한 모델의 학습시간이 2~3배 더 걸림
- 그래프 : 랜덤으로 모델의 파라미터를 초기화하는 것보다 사전학습 모델의 초기값을 사용하였을 때 더 빠르고 높은 정확도에 도달



#01 전이학습의 개념과 필요성

사전 학습된 모델 특징

: 처음부터 학습시키는 대신 소량의 데이터로 특정 작업에 맞춰 미세조정(Fine-tuning)할 수 있어 높은 성능, 빠른 개발 속도, 컴퓨팅 자원 절약

특징

- 높은 초기 성능 및 패턴 인식 : 방대한 데이터를 통해 기본적인 이미지의 특징을 이해하고 있어 초기 성능 우수
- 시간 및 자원 절감 : 처음부터 학습시키는 데 필요한 대규모 인프라와 시간을 줄여줌
- Fine tuning 최적화 : 특정 목적에 맞춰 추가 데이터를 활용해 모델의 가중치를 미세하게 조정해 정확도를 높일 수 있음
- 다양한 Model Architecture : 트랜스포머, CNN(ResNet, VGG 등) 다양한 구조를 기반으로 자연어 처리, 컴퓨터 비전, 음성 AI 등 다양한 산업에 활용됨

#02 전이학습 방법 : 특성 추출 기법

1. 특성 추출 기법

- 특성 추출(feature extractor) : ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후, 마지막의 완전연결층 부분만 새로 학습
- 1) 합성곱층 : 합성곱층과 풀링층 -> 새로운 데이터 통과
- 2) 데이터 분류기(완전연결층) : 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스 분류
-> 합성곱층의 출력을 훈련

#02 전이학습 방법 : 특성 추출 기법

사전 학습된 모델 예시

(1) 자연어 처리 (NLP)

- BERT(Bidirectional Encoder Representations from Transformers) : 문장의 앞뒤 문맥을 동시에 이해하는 양방향 자연어 처리 모델
- GPT(Generative Pre-trained Transformer) : 이전 단어들을 기반으로 다음 단어를 예측하며 문장을 생성하는 언어 모델

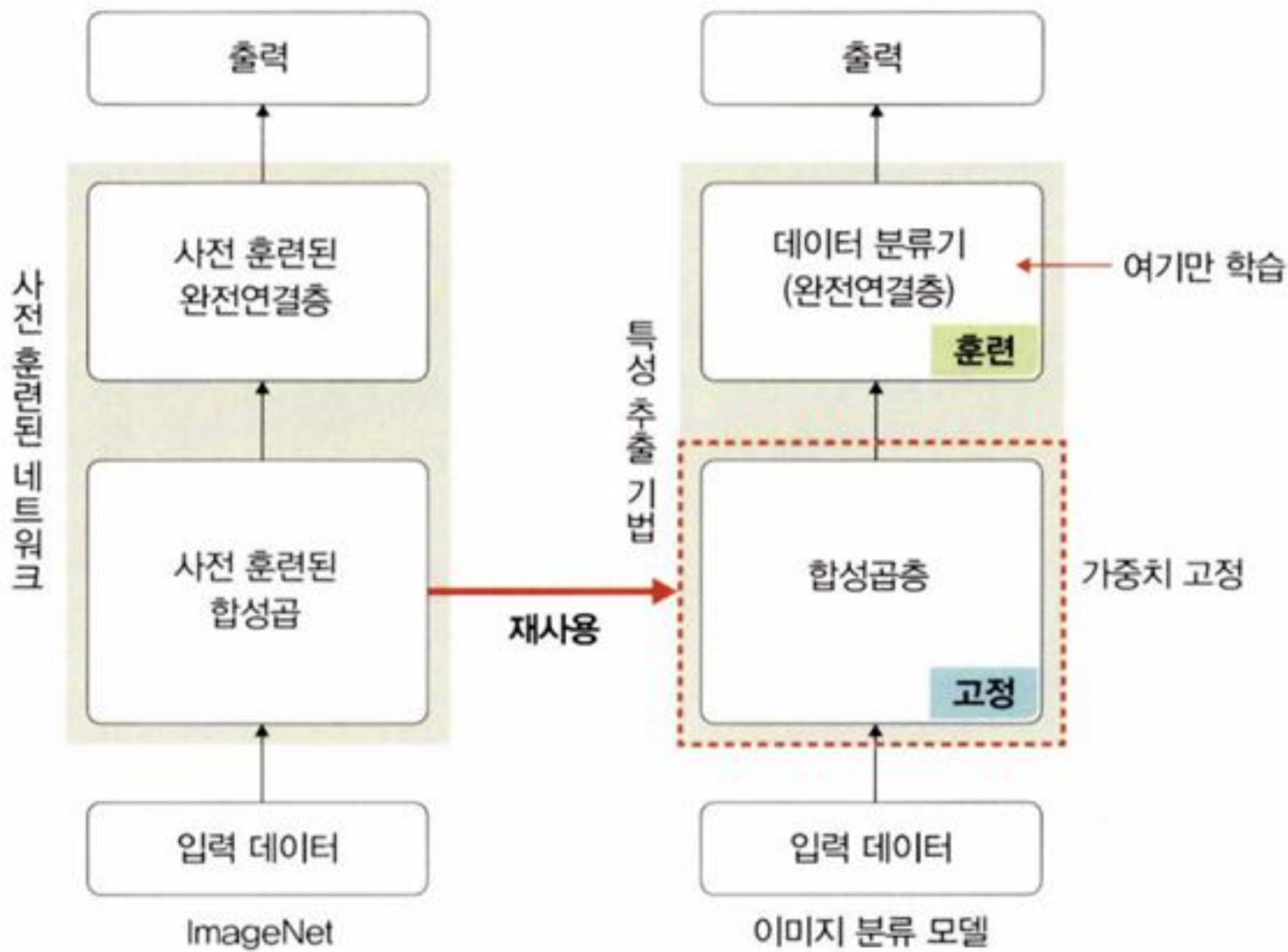
(2) 이미지 처리

- ResNet (Residual Network) : 이전 층의 출력을 더하는 잔차 연결로 깊은 신경망 학습을 안정화한 이미지 모델
- VGG(Visual Geometry Group) : 단순한 3×3 합성곱 층을 깊게 쌓아 특징을 추출하는 이미지 분류 모델

#02 전이학습 방법 : 특성 추출 기법

특성 추출 기법

▼ 그림 5-30 특성 추출 기법



#03 특성 추출 방법 코드

1) 라이브러리 호출

#라이브러리 호출

```
import os
import time
import copy
import glob
import cv2
import shutil

import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```

기본 파이썬 라이브러리

- Os : 파일/폴더 경로
- Time : 실행 시간 측정
- Copy : 객체 복사 (모델 저장 시 중요)
- Glob : 특정 패턴의 파일 찾기
- OpenCV : 오픈소스 컴퓨터 비전 라이브러리
 - Cv2 (OpenCV) : 이미지 처리 라이브러리, resize 및 전처리 등
- Shutil : 파일 복사/이동 (ex. 데이터셋 분할 시)

Pytorch 라이브러리

- Torch : 텐서 연산, GPU 사용
- Torchvision : 이미지 관련 모듈, 데이터셋 및 모델 제공
- Transforms : 이미지 전처리
- Models : 사전학습 모델 불러오기
- Nn : 신경망 layer 정의
- Optim : 최적화 알고리즘

#03 특성 추출 방법 코드

2) 데이터 불러오기 및 데이터 전처리 방법 정의

```
1 data_path = '/content/drive/MyDrive/catanddog/train' # 경로
2
3 transform = transforms.Compose(
4     [
5         transforms.Resize([256, 256]),
6         transforms.RandomResizedCrop(224),
7         transforms.RandomHorizontalFlip(),
8         transforms.ToTensor(),
9     ])
10
11 train_dataset = torchvision.datasets.ImageFolder(
12     data_path,
13     transform=transform
14 )
15
16 train_loader = torch.utils.data.DataLoader(
17     train_dataset,
18     batch_size=32,
19     num_workers=8,
20     shuffle=True
21 )
22
23 print(len(train_dataset))
```

385

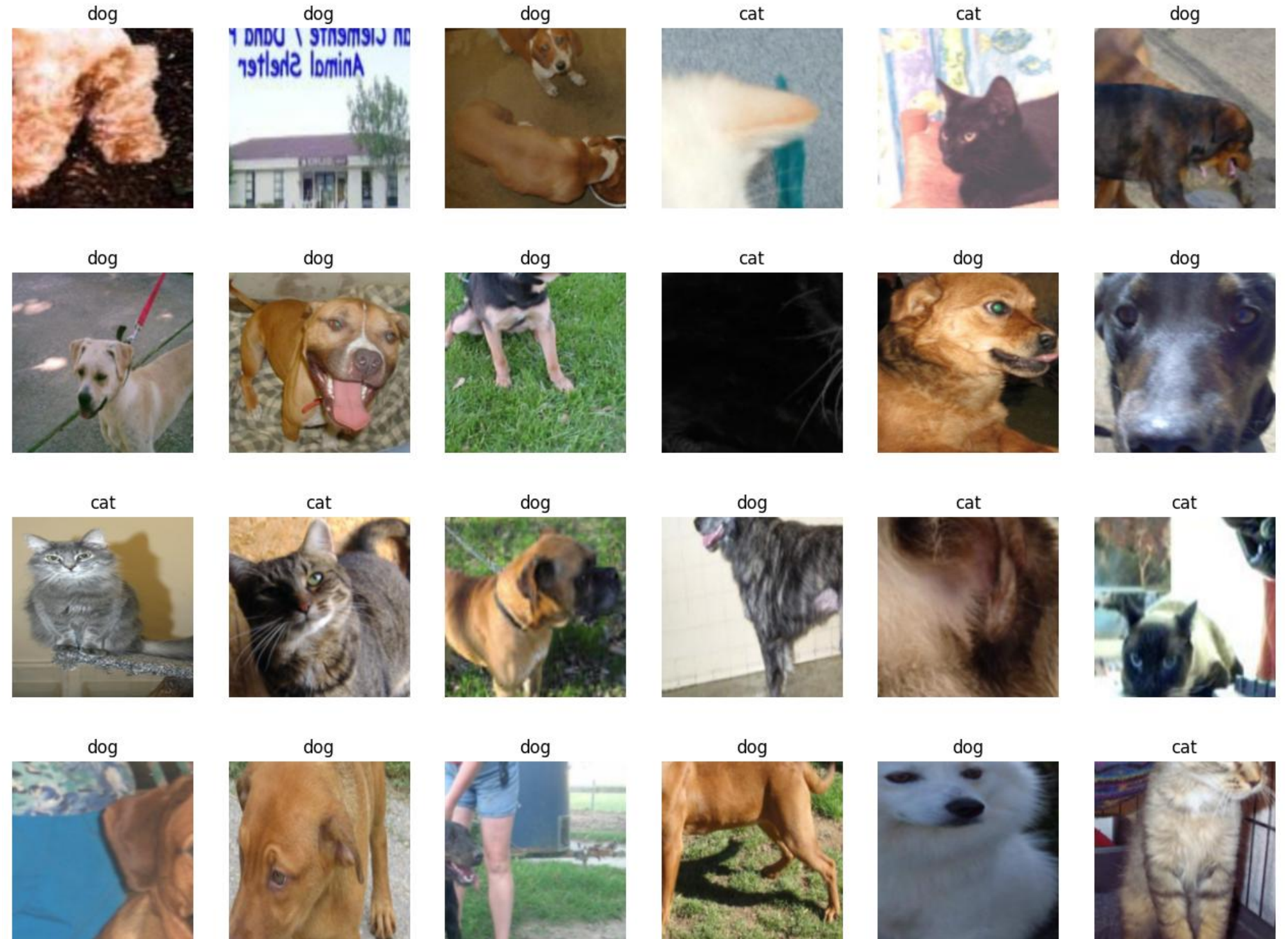
- **Transform** : 여러 변환(transformations)을 함께 묶어 순차적으로 적용
 - 지정된 경로에서 이미지를 불러오고 앞서 정의한 변환을 적용하여 데이터셋을 생성
 - ImageFolder: 주어진 디렉토리 구조에 따라 이미지를 자동으로 라벨링
 - torchvision.transforms: 이미지 데이터를 변환하여 모델(네트워크)의 입력으로 사용할 수 있게 변환
- **데이터로더** : 데이터셋에서 배치(batch) 단위로 데이터를 로드
 - Batch_size = 32 : 한번에 32개의 이미지 로드
 - Num_workers = 8 : 데이터 로딩을 위해 8개의 병렬 작업자(threads)를 사용
 - shuffle=True: 데이터를 무작위로 섞어서 모델이 데이터의 순서에 의존하지 않도록 함

#03 특성 추출 방법 코드

3) 학습에 사용될 이미지 출력

```
1 import numpy as np
2 samples, labels = next(iter(train_loader))
3 classes = {0:'cat', 1:'dog'}
4 fig = plt.figure(figsize=(16,24))
5 for i in range(24):
6     a = fig.add_subplot(4,6,i+1)
7     a.set_title(classes[labels[i].item()])
8     a.axis('off')
9     a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
10 plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- 이전 단계에서 생성한 Data Loader 이용 (이미지 + 라벨)
- Classes : 개와 고양이에 대한 클래스로 구성
- 한번에 24개 이미지를 출력
- 이미지 출력 시 변환
: transpose 필요
Pytorch 이미지 : (채널, 높이, 너비)
Matplotlib : (높이, 너비, 채널)
=> 변환 : (C, H, W) -> (H, W, C)



#03 특성 추출 방법 코드

ResNet18:

- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스의 100만 개 넘는 영상을 이용해 훈련된 신경망
- 입력 제약이 매우 크고, 충분한 메모리가 없으면 학습 속도가 느릴 수 있음

사전 훈련된 모델 내려받기

```
resnet18 = models.resnet18(pretrained=True)
```

pretrained=True로 설정 시 사전 학습된 모델을 사용

다양한 cnn 모델 구조 불러오는 방법 =>
기본적으로 랜덤하게 초기화된 가중치를 가진 상태로 생성한다

사전 훈련된 데이터의 파라미터 학습 유무 지정

```
def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False # 합성곱층과 풀링층을 고정  
  
set_parameter_requires_grad(resnet18)
```

Feature extracting = True 일 때

모든 파라미터에 대해서

gradient 계산 안 함. 즉 학습에서 제외

```
import torchvision.models as models  
resnet18 = models.resnet18()  
alexnet = models.alexnet()  
vgg16 = models.vgg16()  
squeezenet = models.squeezenet1_0()  
densenet = models.densenet161()  
inception = models.inception_v3()  
googlenet = models.googlenet()  
shufflenet = models.shufflenet_v2_x1_0()  
mobilenet_v2 = models.mobilenet_v2()  
mobilenet_v3_large = models.mobilenet_v3_large()  
mobilenet_v3_small = models.mobilenet_v3_small()  
resnext50_32x4d = models.resnext50_32x4d()  
wide_resnet50_2 = models.wide_resnet50_2()  
mnasnet = models.mnasnet1_0()
```


#03 특성 추출 방법 코드

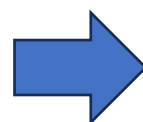
완전연결층 교체

```
resnet18.fc = nn.Linear(512, 2)
```

기존 ResNet18는 1000개 클래스 가지기 때문

모델의 파라미터 값 확인

```
for name, param in resnet18.named_parameters():  
    if param.requires_grad:  
        print(name, param.data)
```



```
fc.weight tensor([[ -0.0042, -0.0266, -0.0104, ...,  0.0439, -0.0022, -0.0176],  
                  [ 0.0339, -0.0358, -0.0022, ..., -0.0359, -0.0094, -0.0382]])  
fc.bias tensor([-0.0117, -0.0045])
```

Requires_grad=True인 파라미터 즉, 현재 학습되는 파라미터 확인

모델 학습 준비

```
model = models.resnet18(pretrained = True) # 모델 객체 생성
```

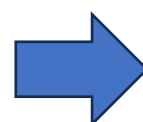
```
for param in model.parameters(): # 합성곱층 가중치 고정  
    param.requires_grad = False
```

```
model.fc = torch.nn.Linear(512, 2) # 마지막 레이어 교체
```

```
for param in model.fc.parameters(): # 완전연결층은 학습  
    param.requires_grad = True
```

```
optimizer = torch.optim.Adam(model.fc.parameters())  
cost = torch.nn.CrossEntropyLoss()  
print(model)
```

모든 레이어 학습 중지



완전 연결층만 학습하도록 함

optimizer에 fc 파라미터만 넣음
즉, 업데이트할 파라미터에서 제외

```
ResNet(  
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)  
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (relu): ReLU(inplace=True)  
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)  
  (layer1): Sequential(  
    (0): BasicBlock(  
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (relu): ReLU(inplace=True)  
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    )  
    (1): BasicBlock(  
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (relu): ReLU(inplace=True)  
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    )  
  )  
)
```


#03 특성 추출 방법 코드

모델 학습을 위한 함수

```
def train_model(model, dataloaders, criterion, optimizer,
                device, num_epochs=13, is_train=True):
    since = time.time()
    acc_history = []
    loss_history = []
    best_acc = 0.0

    에포크 만큼 반복
    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad() 기울기 초기화
            outputs = model(inputs) 순전파 학습
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward() 역전파 학습
            optimizer.step()
```

```
        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

        epoch_loss = running_loss / len(dataloaders.dataset) 평균 오차 계산
        epoch_acc = running_corrects.double() / len(dataloaders.dataset) 평균 정확도 계산

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        torch.save(model.state_dict(), os.path.join('catanddog/', '{0:0=2d}.pth'.format(epoch)))
        print()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))
    return acc_history, loss_history
```

#03 특성 추출 방법 코드

파라미터 학습 결과를 옵티마이저에 전달

```
params_to_update = []  
for name,param in resnet18.named_parameters(): Named_parameters(): (이름, 파라미터) 형태로 가져옴  
    if param.requires_grad == True: 학습 허용된 파라미터만 선택  
        params_to_update.append(param)  
        print("%t",name)  
  
optimizer = torch.optim.Adam(params_to_update)
```

모델 학습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
criterion = nn.CrossEntropyLoss()  
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

#03 특성 추출 방법 코드

테스트 데이터 평가 함수

```
def eval_model(model, dataloaders, device):  
    since = time.time()  
    acc_history = []  
    best_acc = 0.0  
  
    saved_models = glob.glob('catanddog/' + '*.pth')  
    saved_models.sort()  
    print('saved_model', saved_models)  
  
    for model_path in saved_models: 각 epoch 모델을 하나씩 테스트  
        print('Loading model', model_path)  
  
        model.load_state_dict(torch.load(model_path))  
        model.eval()  
        model.to(device)  
        running_corrects = 0  
  
        for inputs, labels in dataloaders: 테스트 반복  
            inputs = inputs.to(device)  
            labels = labels.to(device)
```

```
            with torch.no_grad(): gradient 계산 안 함  
                outputs = model(inputs) 데이터를 모델에 적용한 결과를 outputs에 저장  
  
            _, preds = torch.max(outputs.data, 1) 최댓값이 들어있는 인덱스 반환  
            preds[preds >= 0.5] = 1  
            preds[preds < 0.5] = 0  
            running_corrects += preds.eq(labels).int().sum() 예측 결과와 레이블이 일치하는 것들의 개수 합  
  
        epoch_acc = running_corrects.double() / len(dataloaders.dataset)  
        print('Acc: {:.4f}'.format(epoch_acc))  
  
        if epoch_acc > best_acc:  
            best_acc = epoch_acc  
  
        acc_history.append(epoch_acc.item())  
        print()  
  
    time_elapsed = time.time() - since  
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))  
    print('Best Acc: {:.4f}'.format(best_acc))  
  
    return acc_history
```

#03 특성 추출 방법 코드

평가 함수 적용

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

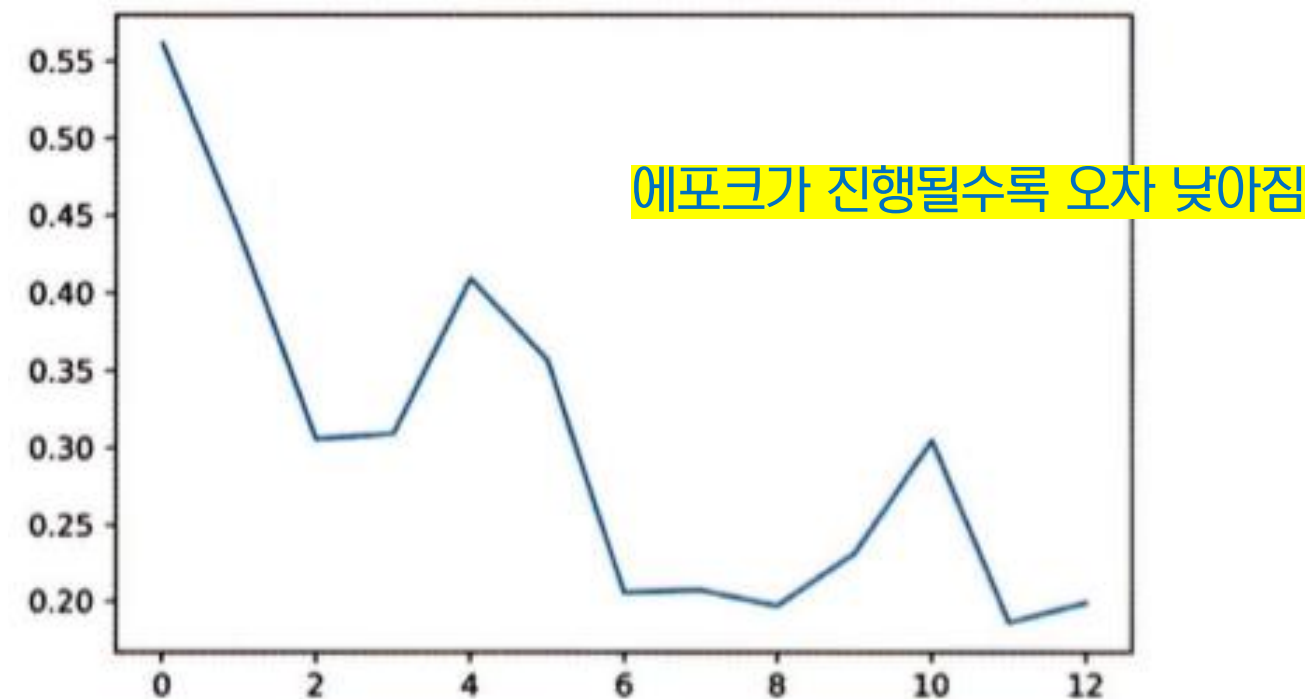
정확도 그래프 확인

```
plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```



오차 그래프 확인

```
plt.plot(train_loss_hist)
plt.show()
```



#03 특성 추출 방법 코드

이미지 출력을 위한 전처리 함수

```
def im_convert(tensor):  
    image=tensor.clone().detach().numpy()  
    image=image.transpose(1,2,0)  
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5)))  
    image=image.clip(0,1)  
    return image
```

텐서 복사 + gradient 제거

데이터를 0과 1 사이의 값으로 제한

구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

개와 고양이 예측 결과 출력

```
classes = {0:'cat', 1:'dog'}  
  
dataiter=iter(test_loader)  
images, labels=next(dataiter)  
output=model(images)  
_, preds=torch.max(output,1)  
  
fig=plt.figure(figsize=(25,4))  
for idx in np.arange(20):  
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])  
    plt.imshow(im_convert(images[idx]))  
    a.set_title(classes[labels[i].item()])  
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),  
                                str(classes[labels[idx].item()]))),  
                color=("green" if preds[idx]==labels[idx] else "red"))  
  
plt.show()  
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```



⇒ 예측이 정확하지 않지만
훈련 데이터와 에포크를
늘리면 성능을 개선할 수 있다.

#04 미세 조정 기법

#1 미세 조정 기법이란?

- 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 **파라미터를 조정**하는 기법
- 특성 추출은 목표 특성을 잘 추출했다는 전제 하에 좋은 성능을 냄
 - > 잘못 추출되었다면 미세 조정 기법으로 가중치를 업데이트해 특성을 다시 추출해야 함.
- 미세 조정 과정에서 많은 연산량 필요 → GPU 사용 권장

#04 미세 조정 기법

#2 미세 조정 기법 사용 전략

1. 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우

: 모델 전체를 재학습 (기존 특징이 잘 안맞는데 데이터가 충분해 과적합 걱정 없음)

2. 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우

: 합성곱층의 뒷부분과 데이터 분류기를 학습 (앞쪽의 일반적인 특징은 그대로 사용하고 뒤쪽의 특화된 특징을 수정)

3. 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우

: 합성곱층의 일부분과 데이터 분류기를 학습 (데이터가 적어 과적합 위험도 있고 유사성이 작아 수정은 필요하기 때문에 적절한 수준에서 일부만 조정할 필요가 있음)

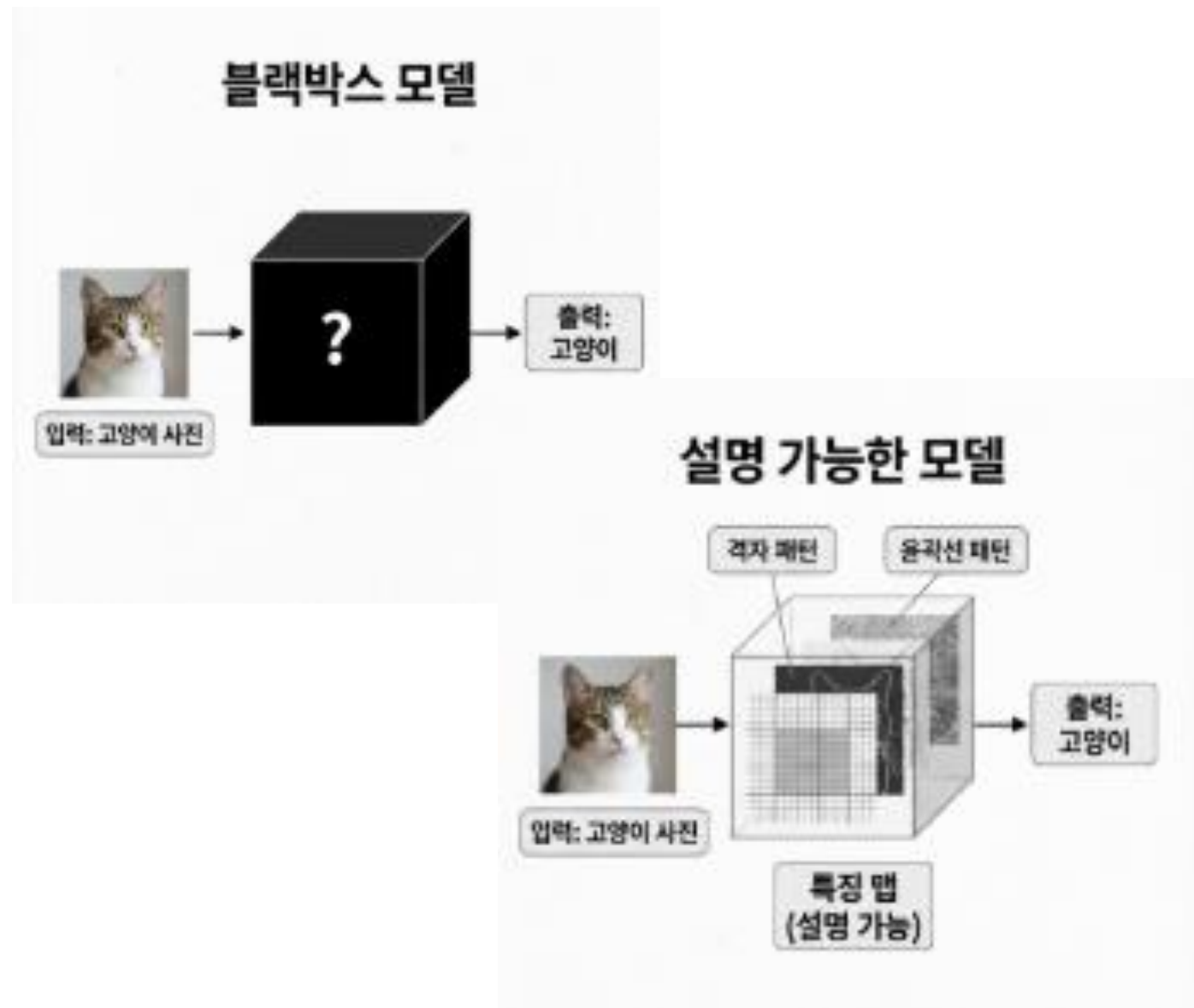
4. 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우

: 데이터 분류기만 학습 (많은 계층 학습 시 과적합 발생할 수 있음)

Explainable CNN



설명 가능한 CNN: 블랙박스를 보이는 모델로



● 설명 가능한 CNN의 목적

- 1 중간 계층의 출력(feature map)을 시각화하여 모델이 무엇을 보고 판단했는지 보여주는 접근
- 2 성능뿐 아니라 해석 가능성과 신뢰성 확보가 목적
- 3 모델의 판단 근거를 이해하고 검증할 수 있게 함

핵심 개념: CNN은 이미지를 그대로 복사하는 것이 아니라 **점점 추상화**하는 과정을 거칩니다.

● 핵심 질문: CNN은 무엇을 보고 분류하는가?

CNN은 이미지 분류에서 성능이 뛰어나지만, 내부 판단 과정을 이해하기 어려워 블랙박스 모델로 불립니다. 결과만 보고는 왜 그렇게 예측했는지 알기 어렵습니다.

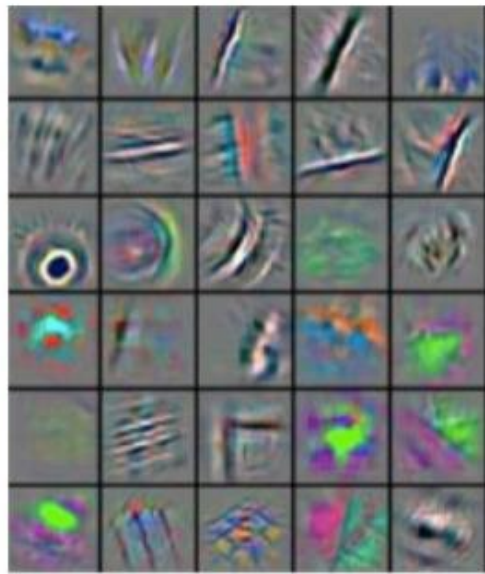
Feature Map과 계층별 특징 변화

CNN의 각 계층에서 특징이 어떻게 변화하는지 시각화

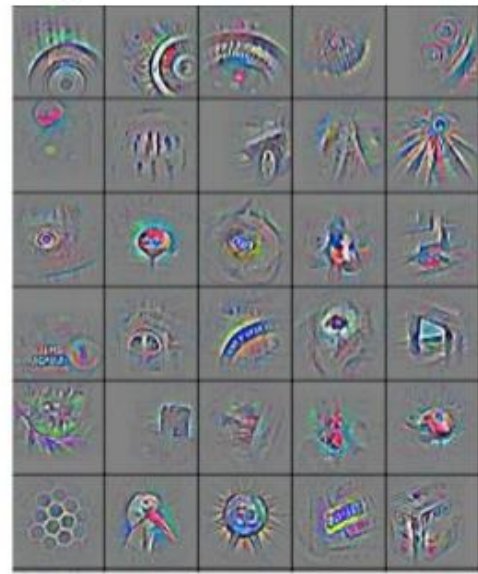
low-level features



mid-level features



high-level features



● Feature Map의 의미

Feature map은 **필터를 입력 이미지에 적용한** 결과입니다. 각 필터는 서로 다른 특징에 반응하며, 이를 통해 CNN이 어떤 정보를 중요하게 보는지 파악할 수 있습니다.

1

초기 계층: 입력과 가까운 계층
원본 이미지 형태가 비교적 유지됨

2

중간 계층: 특징 결합
저수준 특징들이 결합되어 더 유용한 표현

3

깊은 계층: 추상화
분류에 필요한 고차원 특징만 남음

● 핵심 질문: CNN은 무엇을 보고 분류하는가?

CNN은 이미지 분류에서 성능이 뛰어나지만, 내부 판단 과정을 이해하기 어려워 블랙박스 모델로 불립니다. 결과만 보고는 왜 그렇게 예측했는지 알기 어렵습니다.

pixel-level feature → pattern → abstract representation
추상화합니다

CNN은 이미지를 점진적으로

실습 환경과 모델 구조, 전체 과정 개요

CNN 모델 구성과 특성 맵 추출을 위한 전체 실습 흐름

●사용 라이브러리

- **matplotlib**: 시각화 라이브러리
- **PIL / cv2**: 이미지 처리
- **torch / torchvision**: 딥러닝 모델 및 텐서 처리

●예제 모델 구조

Conv(13)

13개의 합성곱층 + 2개의 완전연결층

Dropout

MaxPool

다운샘플링 및 특징 추출

2x2

FC(2)

완전연결층

log_softmax

●최종 출력

클래스 점수 형태로 출력되며, log_softmax를 이용해 확률처럼 해석 가능하게 만들

ReLU, BN,

Not Softmax 이유? 소프트 맥스는 기울기 문제에 취약하기 때문

●모델 객체화

```
model=XAI() # model 이라는 이름의 객체를 생성(인스턴스)
```

```
model.cpu() # 모델에 입력되는 이미지를 넘파이로 받아오는 부분때문에 CPU를 사용하도록 지정
```

```
model.eval() # 테스트 데이터에 대한 모델 평가 용도로 사용
```

●전체 실습 흐름

이미지 준비 → 모델 생성 → 계층 선택 → 출력 추출 → 시각화

●실습 과정 상세

1. 이미지 준비: 고양이 사진 불러오기, 크기 조정
2. 모델 생성: CNN 모델 인스턴스화
3. 계층 선택: 특정 Conv 계층 지정
4. 출력 추출: Hook으로 중간 결과 저장
5. 시각화: Feature map으로 변환

```
class XAI(torch.nn.Module):  
    def __init__(self, num_classes=2): # 클래스가 2개  
        super(XAI, self).__init__()  
  
    self.features = nn.Sequential(  
        nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(256, 512, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2, stride=2),  
        nn.Flatten(),  
        nn.Linear(512, 1000),  
        nn.ReLU(),  
        nn.Linear(1000, num_classes),  
        nn.LogSoftmax(dim=1)  
        )
```

생략....

```
def forward(self, x):  
    x = self.features(x)  
    x = x.view(-1, 512) # 특성 맵에서 추출 후 분류기 거치기 전에 평탄화 작업  
    x = self.classifier(x)  
    return F.log_softmax(x)
```


중간 출력 가져오기: Hook과 LayerActivations

CNN의 특정 계층 출력을 가로채어 중간 특징을 시각화하는 방법

●CNN 구조에서 특정 Layer 선택

Input	입력 이미지 (224x224x3)	활성화
Conv1	64 filters, 3x3 kernel	활성화
Conv2	128 filters, 3x3 kernel	활성화
MaxPool	2x2 pooling	대기
FC	Fully Connected	대기

●Hook의 역할

- 순전파 중 특정 계층 출력 가로채기
- 모델 구조를 바꾸지 않고 중간 결과 확인
- 원하는 계층의 feature map 쉽게 추출

●register_forward_hook() 개념

Input → Layer → Output → Hook 저장

순전파 중 특정 계층의 출력을 가로채어 저장

●LayerActivations 클래스 구현

class LayerActivations: # 중간 출력 저장을 위한 클래스

__init__ model, layer_idx 초기화, activations 저장소 생성
register_hook register_forward_hook()으로 특정 계층 출력 가로채기
get_activations 저장된 중간 출력 반환

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        output = output
        #self.features = output.to(device).detach().numpy()
        self.features = output.detach().numpy()

    def remove(self):
        self.hook.remove()
```

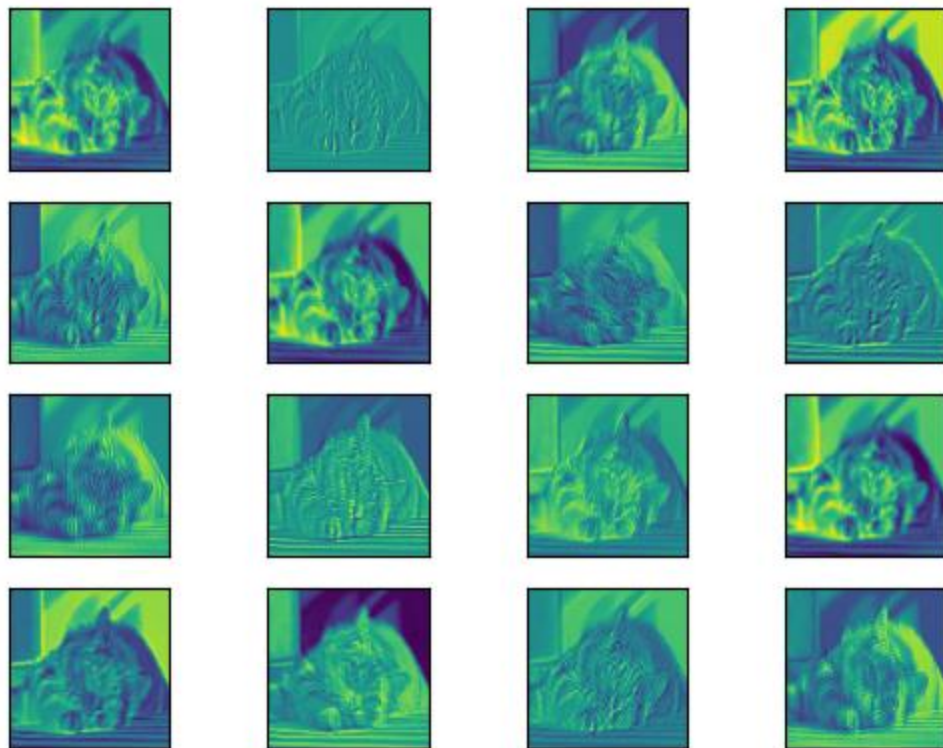

입력 이미지 준비와 첫 번째 계층 특성 맵

이미지 전처리 과정과 1번째 Conv 계층의 feature map 시각화

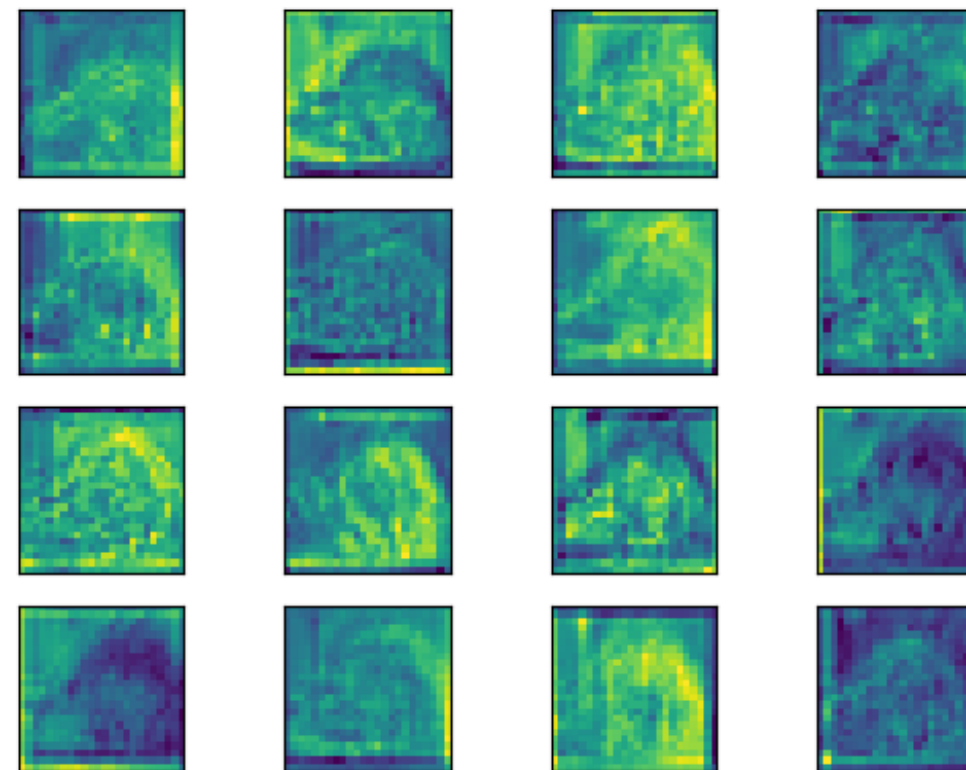
● 입력 이미지 준비 과정

- 1 이미지 호출 - 고양이 사진 불러오기
- 2 크기 조정 - 100x100로 resize
- 3 텐서 변환 - torch tensor로 변환
- 4 배치 차원 추가 - unsqueeze(0) - 1차원 데이터로 변경

● 원본 이미지



● 1번째 Conv 계층 특성 맵 (저수준 특징)



```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload() # chap05/data/cat.jpg 데이터 불러오기

img=cv2.imread("cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)

print(img.shape)
```

1st Layer

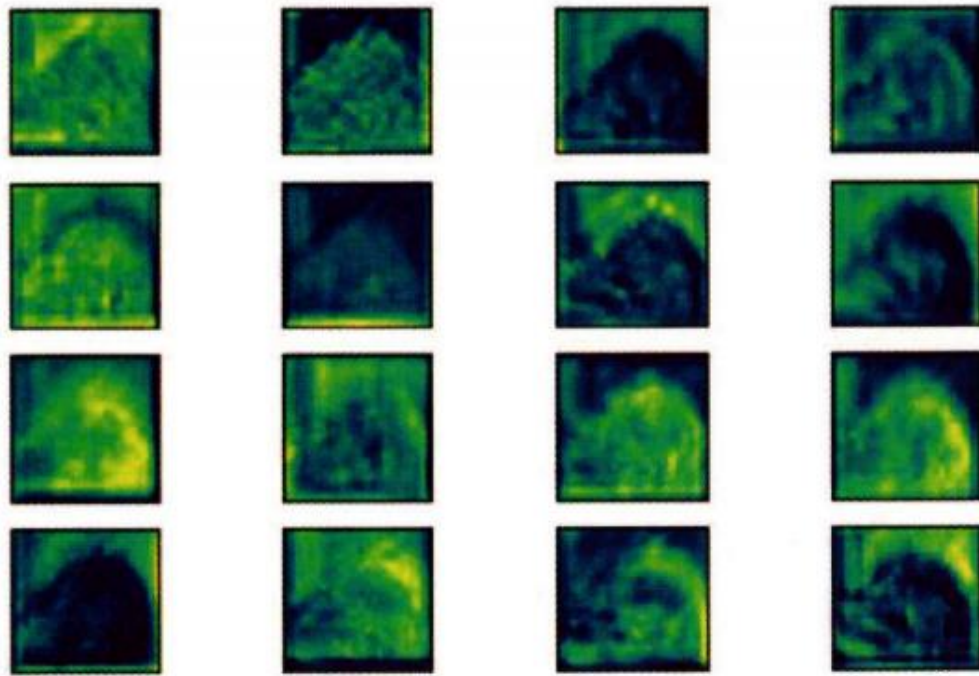
입력과 가까운 계층 = 형태가 남아 있음

● 특징 분석

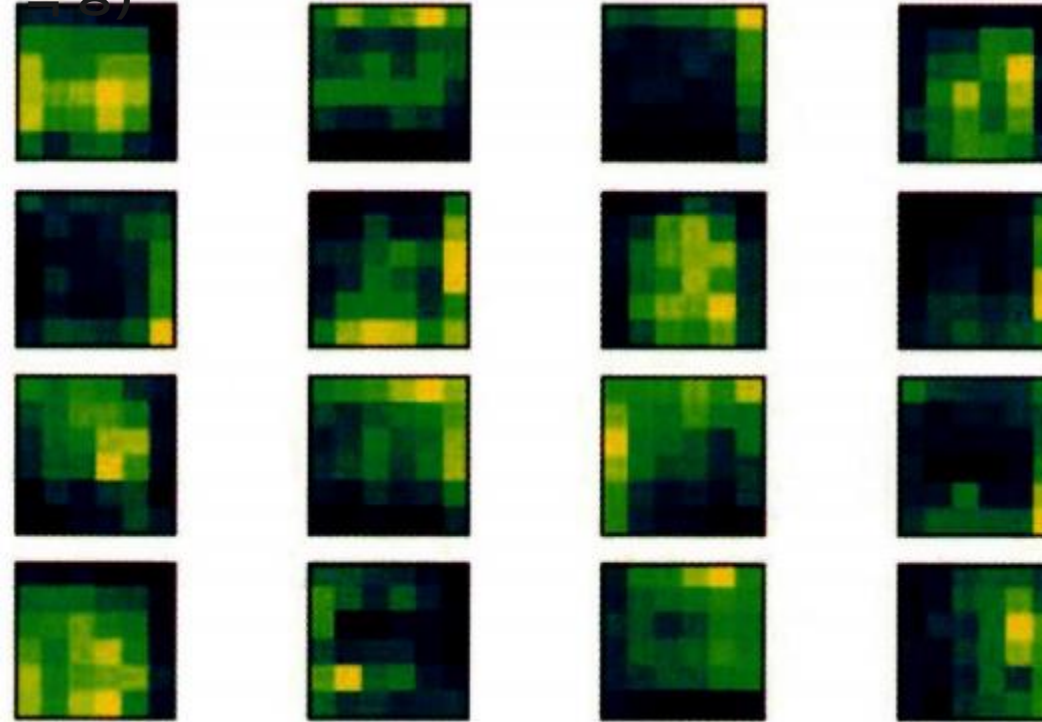
- 원본 형태 유지: 입력과 가까운 계층이라 형태가 선명함
- 저수준 특징: 윤곽선, 밝기, 대비 등 기본 특징
- 시각적 해석: 고양이의 윤곽과 귀 모양이 보임

20,40번째 계층 특성 맵과 해석

● 20번째 Conv 계층 특성 맵 (중간 수준 특징)



● 40번째 계층 특성 맵 (고수준 추상 특징)



● 특성 맵 확인 Code

```
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

```
result = LayerActivations(model.features, 40) # 40번째 Conv2d 특성 맵 확인

model(img)
activations = result.features
```

● 핵심 포인트

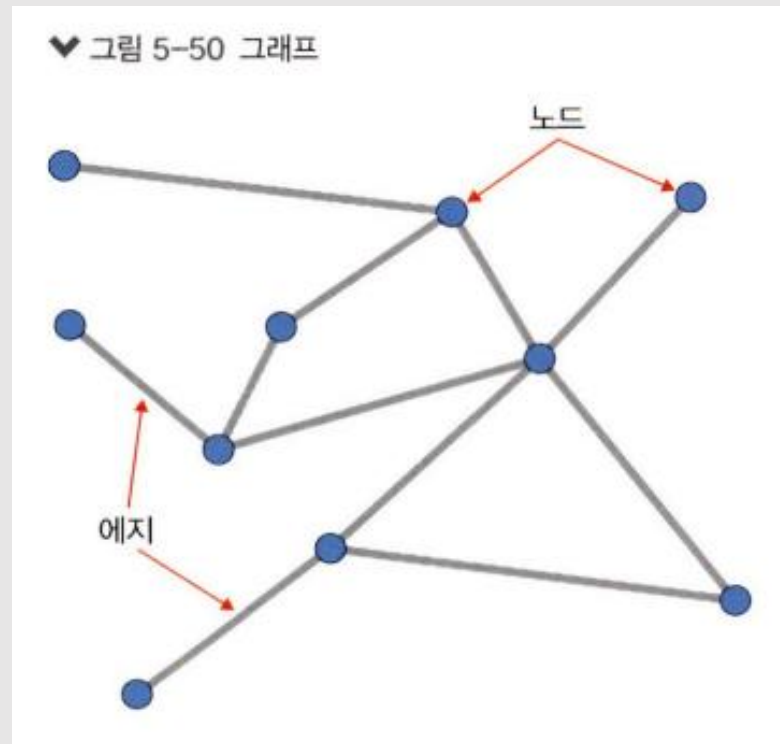
출력층에 가까워질수록 원래 형태를 잃어가고 이미지 특징들만 전달됨

GCN



그래프 개념 정리: 관계 중심의 데이터 구조

그래프의 정의



● 그래프의 구성 요소

- **노드(Node)**: 그래프의 기본 단위, 개체(객체)를 표현 -> 파란색 원(원소)
- **엣지(Edge)**: 노드 간의 관계(연결)를 표현 -> 노드 연결한 선(결합방법)
- 일반적으로 풀고자 하는 문제에 대한 전문가 지식이나 직관 등으로 구성됨

● 그래프의 정의

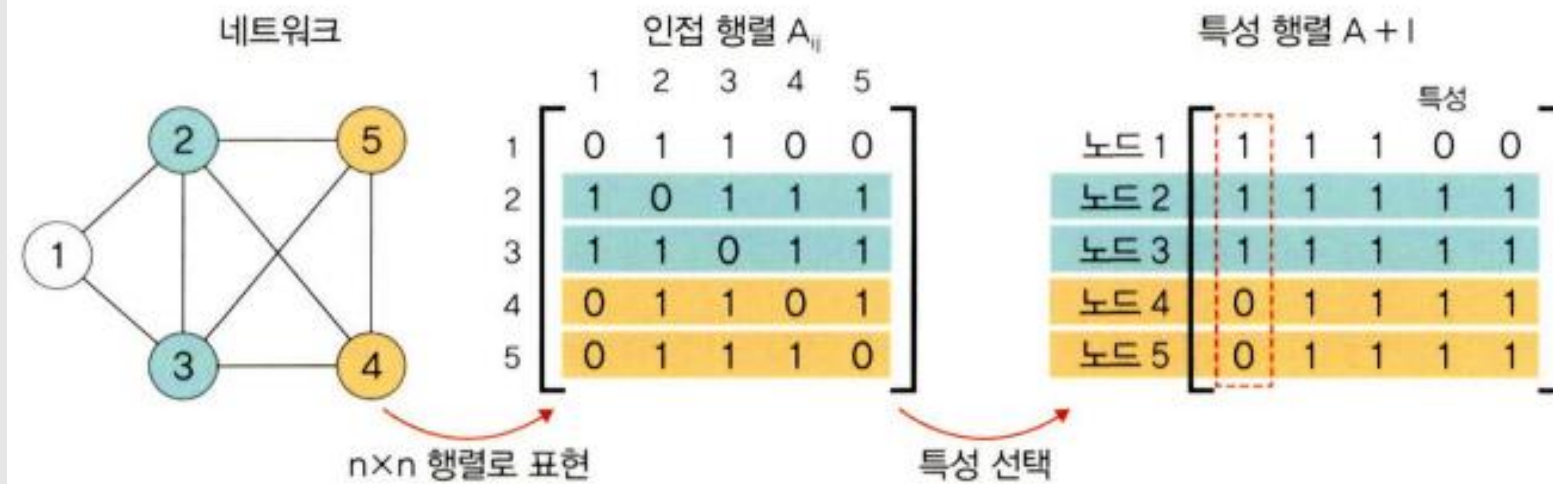
그래프는 **노드(node)**와 **엣지(edge)**로 구성된 데이터 구조로, 객체 간의 관계를 표현합니다.. 방향성이 있거나(directed) 없는(undirected) 엣지로 연결된 노드의 집합

"그래프는 관계를 표현하는 데이터 구조" = 비정형 데이터의 핵심

GNN의 정의와 개념

그래프 신경망(GNN)의 핵심 아이디어

▼ 그림 5-51 특성 행렬⁵



● GNN의 정의

그래프 구조에서 사용하는 신경망, 그래프 데이터에 대한 표현은 두 단계로 이루어짐

● 인접 행렬 (Adjacency Matrix)

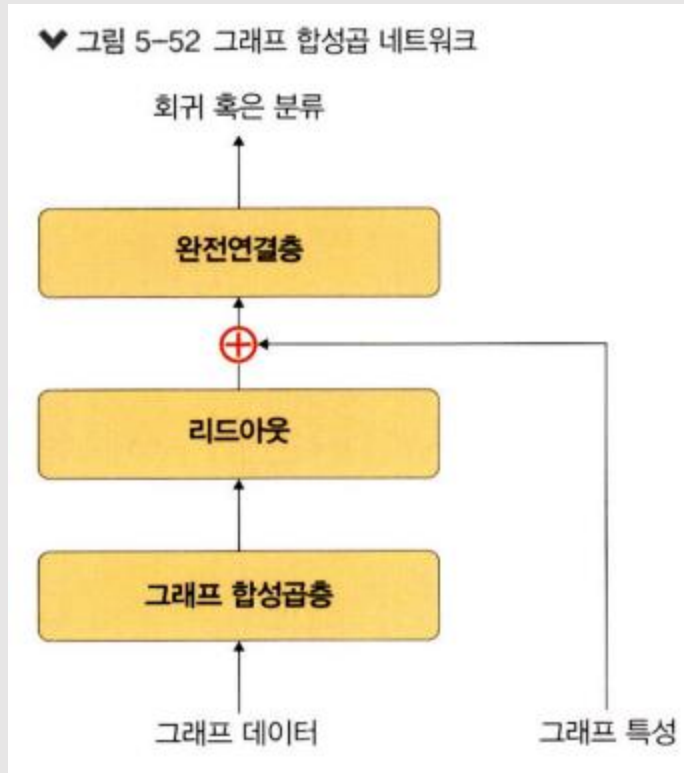
- 노드 n 개를 $n \times n$ 행렬로 표현
 - 인접 행렬 내의 값은
' A_{ij} 는 i 와 j 의 관련성 여부'를 만족하는 값으로 채워줌
- > 컴퓨터가 이해하기 쉽게 그래프로 표현

● 특성 행렬 (feature matrix)

- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용
- 각 입력 데이터에 이용할 특성을 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미함
-> 첫 번째 행은 첫 번째 노드의 특성 값
- 이제 노드 1, 2, 3, 4에 대한 특성을 한눈에 파악하기 쉽게 표현됨

GNN의 구조

CNN의 합성곱 개념을 그래프 데이터로 확장한 모델



● GNN의 활용

SNS에서 관계 네트워크, 학술 연구에서 인용 네트워크
3D Mesh

● Leadout

특성 행렬을 하나의 벡터로 변환하는 함수

전체 노드의 특성 벡터에 대해 평균을 구하고
그래프 전체를 표현하는 하나의 벡터를 생성함

● 특성 행렬 (feature matrix)

그래프 합성곱층을 이용한 그래프 형태의 데이터는
행렬 형태의 데이터로 변환되어
딥러닝 알고리즘을 적용할 수 있도록 함

THANK YOU

